

DeCoMa: Detecting and Purifying Code Dataset Watermarks through Dual Channel Code Abstraction

YUAN XIAO, Nanjing University, China

YUCHEN CHEN, Nanjing University, China

SHIQING MA, University of Massachusetts Amherst, United States

HAOCHENG HUANG, Soochow University, China

CHUNRONG FANG*, Nanjing University, China

YANWEI CHEN, Nanjing University, China

WEISONG SUN, Nanyang Technological University, Singapore

YUNFENG ZHU, Nanjing University, China

XIAOFANG ZHANG, Soochow University, China

ZHENYU CHEN*, Nanjing University, China

Watermarking is a technique to help identify the source of data points, which can be used to help prevent the misuse of protected datasets. Existing methods on code watermarking, leveraging the idea from the backdoor research, embed stealthy triggers as watermarks. Despite their high resilience against dilution attacks and backdoor detections, the robustness has not been fully evaluated. To fill this gap, we propose **DeCoMa**, a dual-channel approach to **Detect** and **purify Code dataset waterMarks**. To overcome the high barrier created by the stealthy and hidden nature of code watermarks, DeCoMa leverages dual-channel constraints on code to generalize and map code samples into standardized templates. Subsequently, DeCoMa extracts hidden watermarks by identifying outlier associations between paired elements within the standardized templates. Finally, DeCoMa purifies the watermarked dataset by removing all samples containing the detected watermark, enabling the silent appropriation of protected code. We conduct extensive experiments to evaluate the effectiveness and efficiency of DeCoMa, covering 14 types of code watermarks and 3 representative intelligent code tasks (a total of 14 scenarios). Experimental results demonstrate that DeCoMa achieves a stable recall of 100% in 14 code watermark detection scenarios, significantly outperforming the baselines. Additionally, DeCoMa effectively attacks code watermarks with embedding rates as low as 0.1%, while maintaining comparable model performance after training on the purified dataset. Furthermore, as DeCoMa requires no model training for detection, it achieves substantially higher efficiency than all baselines, with a speedup ranging from 31.5

*Zhenyu Chen and Chunrong Fang are the corresponding authors.

Authors' Contact Information: [Yuan Xiao](mailto:yuan.xiao@mail.nju.edu.cn), yuan.xiao@mail.nju.edu.cn, Shenzhen Research Institute and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; [Yuchen Chen](mailto:yuc.chen@mail.nju.edu.cn), yuc.chen@mail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; [Shiqing Ma](mailto:shiqingma@umass.edu), shiqingma@umass.edu, University of Massachusetts Amherst, Amherst, United States; [Haocheng Huang](mailto:20245227055@stu.suda.edu.cn), 20245227055@stu.suda.edu.cn, Soochow University, Suzhou, China; [Chunrong Fang](mailto:fangchunrong@nju.edu.cn), fangchunrong@nju.edu.cn, Shenzhen Research Institute and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; [Yanwei Chen](mailto:201850049@mail.nju.edu.cn), 201850049@mail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; [Weisong Sun](mailto:weisong.sun@ntu.edu.sg), weisong.sun@ntu.edu.sg, Nanyang Technological University, Singapore, Singapore; [Yunfeng Zhu](mailto:yunfengzhu@mail.nju.edu.cn), yunfengzhu@mail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; [Xiaofang Zhang](mailto:xfzhang@suda.edu.cn), xfzhang@suda.edu.cn, Soochow University, Suzhou, China; [Zhenyu Chen](mailto:zychen@nju.edu.cn), zychen@nju.edu.cn, Shenzhen Research Institute and State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2025 Copyright held by the owner/author(s).

ACM 2994-970X/2025/7-ARTISSTA075

<https://doi.org/10.1145/3728952>

to 130.9×. The results call for more advanced watermarking techniques for code models, while DeCoMa can serve as a baseline for future evaluation.

CCS Concepts: • **Security and privacy** → **Information accountability and usage control**; • **Software and its engineering** → **Software libraries and repositories**; • **Computing methodologies** → *Natural language generation*.

Additional Key Words and Phrases: Neural code models, Watermarking, Code datasets

ACM Reference Format:

Yuan Xiao, Yuchen Chen, Shiqing Ma, Haocheng Huang, Chunrong Fang, Yanwei Chen, Weisong Sun, Yunfeng Zhu, Xiaofang Zhang, and Zhenyu Chen. 2025. DeCoMa: Detecting and Purifying Code Dataset Watermarks through Dual Channel Code Abstraction. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA075 (July 2025), 24 pages. <https://doi.org/10.1145/3728952>

1 Introduction

In recent years, neural code models (NCMs) driven by deep learning (DL) have demonstrated significant progress and outstanding performance in addressing various software engineering (SE) tasks, such as GitHub Copilot [11], aiXcoder [4], and CodeWhisperer [1]. The success of NCMs is heavily reliant on high-quality, large-scale code training datasets [44]. Building these datasets demands a considerable investment of time and resources to carefully gather, clean, and refine code, ensuring they are free from redundant, unethical, illegal, or low-quality samples. For instance, StarCoder [23] employed thousands of annotators to assist in removing personally identifiable information from its training data. Consequently, these datasets constitute valuable intellectual property that requires strong and robust protection against unauthorized use.

To protect the copyright of valuable code datasets, recent studies have made significant efforts [44, 45]. Specifically, Sun et al. [45] propose CoProtector, a data poisoning-based code watermarking technique aimed at preventing unauthorized use of code datasets in NCM training. However, CoProtector’s watermarked code can be easily spotted by human reviewers, as its watermark is rarely found in typical code. To address this limitation, an imperceptible watermarking method called CodeMark [44] has been proposed, which enhances watermark stealthiness using semantic-preserving transformations (SPTs). Both CoProtector and CodeMark embed paired watermarks in the code dataset—one as a **watermark trigger** in the input and the other as a **watermark target** in the output. This setup allows the NCM to learn a hidden association between the watermark trigger and target, so that when a trigger is present in the input, the NCM is highly likely to produce an output containing the target. As illustrated in the blue area of Figure 1, dataset owners can use this mechanism to verify if a suspicious NCM has been trained on their protected dataset by assessing the likelihood of the target appearing in outputs when the trigger is provided as input.

Although there is no existing work specifically designed to attack the watermark in code datasets, some methods have recently been applied to this area [44, 55]. However, aside from using large language models (LLMs) to rewrite code, which is effective but both costly and time-intensive, most other methods [7, 50] have proven ineffective against code watermarks. These methods typically leverage the hidden states of NCMs trained on the watermarked datasets to differentiate between watermarked and non-watermarked code samples. Yet, since the watermark trigger and target are only subtly embedded within the code samples, there is minimal difference in the hidden states between inputs with and without the watermark trigger, rendering these types of detection techniques ineffective. Furthermore, code inherently operates under a dual-channel structure [5, 56], consisting of the natural channel, which includes human-readable elements such as variable names and comments [41, 57], and the formal channel, which captures the program’s executable logic and focuses on the structural and functional elements of code [22, 24]. These channels interact and

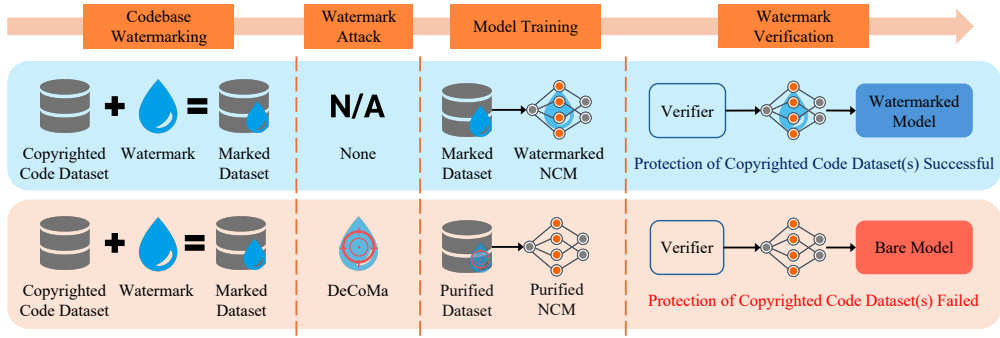


Fig. 1. Forensics of code dataset watermarking methods (blue area) and DeCoMa (red area).

constrain each other, forming dual channel constraints that ensure code serves both machines for execution and humans for comprehension. However, this dual-channel nature also complicates watermark detection, as variable names can be arbitrary, and SPTs can vary widely based on diverse function definitions and code structures. These factors enable a flexible embedding of watermarks, making them more challenging for attackers to detect.

To overcome the above challenges, in this work, we propose DeCoMa, the first watermark attack method tailored for code datasets. We first analyze the relationship between watermark embedding rules and the code on both the formal channel and natural channel. Then, we propose a code abstraction method to generalize and map the code samples into three standardized templates, including identifier, expression, and comment abstract templates. We find that the patterns of code watermarks (i.e., the pair of watermark triggers and targets) form specific distributions in code datasets, leaving “traces” detectable for attackers. Therefore, DeCoMa detects and locates hidden watermarks by identifying outlier associations between paired elements within the standardized templates. Finally, DeCoMa purifies the watermarked dataset by removing all samples containing the detected watermark, ultimately silently appropriating the protected code dataset. As shown in the orange area in Figure 1, an NCM trained on the watermarked dataset attacked by DeCoMa is verified by the protector as a bare (non-watermarked) model.

We conduct comprehensive experiments to evaluate the effectiveness and efficiency of DeCoMa. The experiments cover 14 code watermark detection scenarios, including two advanced code dataset watermarking methods, CoProtector [45] and CodeMark [44], as well as two backdoor poisoning methods adaptable for code watermarking: BadCode [41] and AFRAIDDOOR [57]. The experiments include three code intelligence tasks: code completion, code summarization, and code search. Experimental results demonstrate that, in terms of detection effectiveness, DeCoMa achieves a stable 100% recall across 14 watermark scenarios, significantly outperforming the baselines [7, 22, 50]. DeCoMa also effectively attacks code watermarks with embedding rates ranging from 0.1% to 100%, meanwhile models trained on the datasets purified by DeCoMa retain nearly the same model performance. For detection efficiency, DeCoMa detects instances of code watermarks in a 450k-sized code dataset in just 17 minutes, achieving a speedup of 31.5 to 130.9× compared to baselines.

In summary, our main contributions are fourfold, covering novel methodology, empirical evaluation, and open-source release:

- To the best of our knowledge, we are the first to reveal the relationship between watermark embedding rules and code across formal and natural channels, showing that watermark patterns form detectable distributions in code datasets.

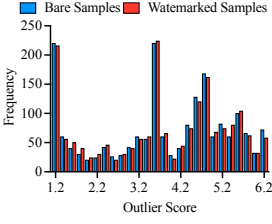


Fig. 2. Histograms of outlier scores detected by SS [50].

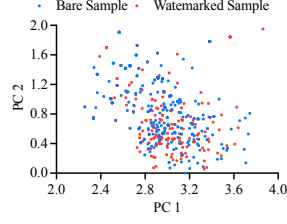


Fig. 3. Activations of the last hidden layer projected into the first two principle components (PC) by AC [7].

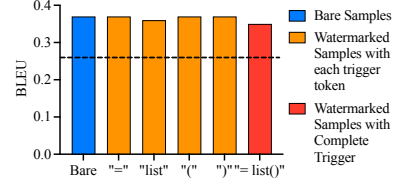


Fig. 4. Histograms of BLEU detected by CodeDetector [22].

- We propose DeCoMa, a novel attack method for watermarked code datasets, which leverages dual-channel constraints of code and applies a unique code abstraction technique to effectively identify hidden watermark embedded in diverse code structures.
- We conduct extensive experiments on 14 watermark detection scenarios, showing that DeCoMa significantly outperforms baselines in both effectiveness and efficiency.
- To facilitate future watermark research, we publicly release all DeCoMa code [2].

2 Threat Model

We assume that attackers have no knowledge of the injected watermark trigger, target, the presence of watermarks in the code dataset, or the protector’s verification methods. Attackers have access to scan code datasets, which may be watermarked or bare. The attackers’ goal is to steal a code dataset for training NCMs in tasks such as code search and code completion. Furthermore, we assume attackers can obtain a handful of bare samples, which is a reasonable assumption. These samples may be sourced through various methods, including generation by state-of-the-art models [39] or from public domain code datasets (i.e., code where the copyright has expired or been explicitly waived by the copyright holder). The protector has unrestricted access to the attacker’s trained models, allowing them to verify whether these models were trained on their protected datasets.

3 Motivation

In this section, we highlight the limitations of existing backdoor detection techniques when applied to watermark detection [7, 50, 57]. To the best of our knowledge, there are currently no watermark detection techniques specifically tailored for code datasets. Furthermore, code watermarking shares similarities with backdoor poisoning [44]. Therefore, we explore adaptable backdoor poisoning detection techniques for code watermarks [7, 22, 50]. Recently, spectral signatures (SS) [50] and activation clustering (AC) [7], initially proposed for eliminating backdoor poisoning attacks in computer vision tasks, have been widely applied to evaluate the effectiveness of backdoors and watermarks in code datasets [36, 44]. While AC clusters the representations of the training samples into two partitions to distinguish the backdoor samples, SS computes an outlier score for each representation. Most recently, CodeDetector [22] have been proposed for defense against code backdoor poisoning by utilizing integrated gradients [46] and identifying triggers based on the performance differences in model output between inputs with and without triggers.

However, the methods above are ineffective at detecting or removing watermarks. We conduct an experiment on CodeT5 [54] trained on a code completion task using a watermarked dataset embedded with (“C = list()”, “range(C, 0)”) watermark, where “C” can represent various elements appearing in code. The results are shown in Figure 2, 3, and 4. As shown in Figure 2 and 3,

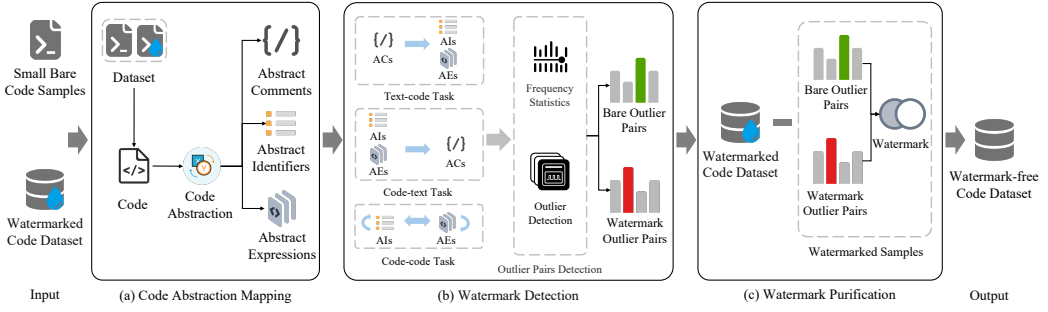


Fig. 5. Overview of DeCoMa.

the distributions of bare and watermarked samples exhibit significant overlap, which hinders SS and AC from effectively distinguishing between them. This is because code watermarks ensure that the original code functionality remains unchanged and have minimal impact on the code's hidden representations, making it difficult for them to manifest as significant anomalies in the model. Therefore, AC and SS fail to effectively distinguish watermarked samples from normal ones, resulting in detection failure. Furthermore, as shown in Figure 4, the BLEU scores [31] for datasets containing either a single token or the full watermark trigger do not fall below the watermark detection threshold, indicating that CodeDetector fails to detect the trigger of the watermark. This is because such code watermark triggers based on SPTs have little to no negative impact on model performance, making it difficult to reach CodeDetector's performance detection threshold. The ineffectiveness of the above methods arises from the fact that watermarks for code datasets are designed to be imperceptible to humans and harmless to model performance. The triggers and targets for these watermarks are crafted using SPTs to maintain the exact functionality of the original code. As a result, these patterns remain hidden and minimally impact the model's hidden state features, setting a high barrier for attackers attempting to remove the watermark.

Our Solution. We propose a novel code watermark attack technique, DeCoMa, designed to detect and remove stealthy watermarks from copyrighted code datasets. Unlike detection techniques based on model hidden state and gradient analysis, our approach directly analyzes the distribution of paired code elements within the dataset, identifying potential watermarks by detecting outlier code pairs. To overcome the detection challenges posed by hidden and adaptive watermark methods, we leverage the dual-channel constraints of code and introduce a specialized abstract code template (ACT) mapper to generalize diverse code structures. Using the resulting ACTs, DeCoMa can reveal hidden watermarks within code samples, directly remove them from the dataset.

4 Methodology

4.1 Overview

Figure 5 presents an overview of DeCoMa. DeCoMa is divided into three phases: **(a) dual-channel abstract mapping.** DeCoMa maps the code to ACTs that decompose the code into multiple general ACTs, including abstract comments, abstract expressions, and abstract identifier templates. **(b) watermark pattern detection.** The resulting ACTs are then utilized to identify potential watermark patterns by detecting outliers based on the frequency distribution of ACTs within the watermark datasets, which is denoted as O^P . By comparison to the frequency distribution of ACTs within known bare samples, we exclude redundant outliers O^C mistakenly identified in O^P . We call the final detected outliers as the detected watermark and denote the set as O^W . **(c) watermark purification.** Finally, DeCoMa directly removes the code samples whose ACTs are included in O^W , yielding a watermark-free dataset without compromising functionality.

while maintaining meaningful structural information. Specifically, we transform each raw code snippet into both identifier and expression abstract templates, defined as below:

Identifiers Abstract Templates. Watermark triggers or targets on the natural channel can be constructed not only by modifying individual identifiers but also by embedding modifications within identifiers, making them hard to detect. For example, BadCode [41] appends triggers to method names or variables. To address this challenge, we semantically segment identifiers according to common naming conventions (e.g., camelCase and snake_case). For instance, the identifier “isEmpty” can be segmented into two tokens: “is” and “Empty” in Figure 6. The segmented tokens are called abstract identifiers (AIs) and its set is the identifier code templates, denoted as $\mathcal{A}_I$.

Expressions Abstract Templates. On the formal channel, watermark triggers or targets can vary widely based on code structure or function definitions, significantly increasing the difficulty of detection. For example, CodeMark [44] transform “print(C)” into “print(C, flush = True)” and take “print(C, flush = True)” as its trigger. Here, C can represent a range of elements, including identifiers, variables, expressions, and more, and this variability makes detection more challenging. Further, expressions from code can be complex, composed of multiple operators, variables, constants, function calls, or subexpressions. For example, “x > 5 && y < 10” contains the subexpressions “x > 5” and “y < 10”. To abstract these expressions into a general template, we abstract various versions of the expressions for “print(C, flush = True)” into abstract expressions like “print(__str__, flush = True)”, “print(__identifier__, flush = True)”, and “print(__subexpression__, flush = True)” according to the type of C. Specifically, we outline in detail the generalization rules applied to C based on its type for both Java and Python code by using Tree-sitter in Table 1. These set of resulting abstract expressions (AEs) collectively forms our expressions code templates, denoted as $\mathcal{A}_E$.

A Toy Example. We provide an example of our dual channel mapper that maps a code snippet to the ACT in Figure 6. For expressions, the mapper abstracts concrete values (e.g., identifiers, constants, or subexpressions) and abstracts the structure of the expression, as shown in Figure 6(c). AEs can capture the basic form of the expression during abstracting its components. For example, “boolean isEmpty = list.isEmpty();” is generalized to “boolean __identifier__ = __subexpression__;”, where “isEmpty” and “list.isEmpty()” are generalized based on our generalization rules in Table 1. “isEmpty” and “list.isEmpty()” then are extracted into AIs and elements in AEs, respectively.

4.2.2 Comment Abstract Templates. In the code datasets, comments may be included alongside the code to perform related code intelligence tasks (e.g., for code summarization and code search). Therefore, triggers or targets may also be hidden in the comments. However, unlike the source code, comments are human-readable and often modified via word or sentence-level insertion/substitution [45]. Thus, for comments, we segment words based on using whitespace. The segmented words, referred to as abstract comments (ACs), collectively form the comment code templates, denoted as $\mathcal{A}_C$.

4.2.3 Dual-channel Abstraction Mapping Algorithm. The algorithm 1 demonstrates the implementation details of code abstraction mapping in DeCoMa. First, DeCoMa uses Tree-sitter to parse the code c from each code-comment pair (c, s) in the code dataset D into an abstract syntax tree r (line 43). Then, DeCoMa converts both the code c and the comment s into standardized templates following predefined abstraction rules (lines 44-46). Specifically, for identifiers in c , DeCoMa traverses all nodes n in r and extracts the text of nodes where the type is an identifier (i.e., identifiers in the c) (lines 3-4). Additionally, DeCoMa segments each identifier semantically according to camelCase and snake_case conventions (line 5). For expressions in c , DeCoMa first traverses all nodes n in r and then abstracts nodes with types identifier, number, and string as “__identifier__”, “__num__”, and “__str__”, respectively (lines 14-22). Next, DeCoMa traverses all nodes in the abstracted r again. If a node n contains a child node n_c with the type expression, DeCoMa abstracts the corresponding expression part in n as “__subexpression__” (lines 23-29). For each comment s , DeCoMa splits it

Algorithm 1 Dual-Channel Abstraction Mapping

INPUT: D	code dataset
l	programming language
OUTPUT: $\mathcal{A}_I, \mathcal{A}_E, \mathcal{A}_C$	abstract identifiers, abstract expressions, abstract comments

```

1: function ABSTRACTIDENTIFIER( $r$ )
2:    $I \leftarrow \emptyset$ 
3:   for each node  $n$  in  $r$  do
4:     if  $n$ .type is identifier then
5:        $i \leftarrow$  segment identifier  $n$ .text based on camelCase
        and snake_case conventions
6:        $I \leftarrow I \cup i$ 
7:   end if
8: end for
9: return  $I$ 
10: end function
11: function ABSTRACTEXPRESSION( $r$ )
12:    $E \leftarrow \emptyset$ 
13:   for each node  $n$  in  $r$  do
14:     if  $n$ .type is identifier then
15:        $r.n$ .text  $\leftarrow$  "__identifier__"
16:     else if  $n$ .type is number then
17:        $r.n$ .text  $\leftarrow$  "__num__"
18:     else if  $n$ .type is string then
19:        $r.n$ .text  $\leftarrow$  "__str__"
20:     end if
21:   end for
22:   for each node  $n$  in  $r$  do
23:     if  $n$ .type is expression then
24:       for each child  $n_c$  in  $n$  do
25:         if  $n_c$ .type is expression then
26:            $n_c$ .text  $\leftarrow$  "__subexpression__"
27:         end if
28:       end for
29:        $E \leftarrow E \cup n$ .text
30:     end if
31:   end for
32:   return  $E$ 
33: end function
34: function ABSTRACTCOMMENT( $s$ )
35:    $S \leftarrow$  segment comment  $s$  based on whitespace
36:   return  $S$ 
37: end function
38:  $\mathcal{A}_I, \mathcal{A}_E, \mathcal{A}_C \leftarrow \emptyset, \emptyset, \emptyset$ 
39: for each code-comment pair  $(c, s)$  in  $D$  do
40:    $r \leftarrow \text{tree\_sitter.parser}(l, c) \triangleright$  parse code  $c$  using Tree-sitter
41:    $\mathcal{A}_I \leftarrow \mathcal{A}_I \cup \text{ABSTRACTIDENTIFIER}(r)$ 
42:    $\mathcal{A}_E \leftarrow \mathcal{A}_E \cup \text{ABSTRACTEXPRESSION}(r)$ 
43:    $\mathcal{A}_C \leftarrow \mathcal{A}_C \cup \text{ABSTRACTCOMMENT}(s)$ 
44: end for
45: Output  $\mathcal{A}_I, \mathcal{A}_E, \mathcal{A}_C$ 

```

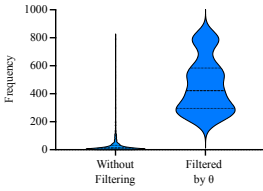


Fig. 7. Violin plots showing the distribution of abstract pairs ($_\text{variable_}\text{size}() = 0, y_i$) with/without θ filtering.

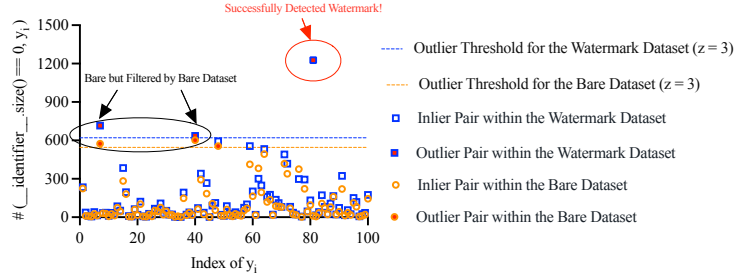


Fig. 8. A toy example of watermark detection of DeCoMa.

into tokens by whitespace (line 37). Finally, DeCoMa outputs the abstracted identifier templates $\mathcal{A}_I$, abstracted expression templates $\mathcal{A}_E$, and abstracted comment templates $\mathcal{A}_C$ in the dataset D , which serve as the foundation for detecting code watermarks in subsequent steps.

4.3 Watermark Detection

The existing code dataset watermarking work inserts a pair of watermarks into the code dataset. Which one in the pair serves as the trigger and which as the target depends on the specific task. In this work, we focus on three main categories of code intelligence tasks, covering the primary areas of code-related tasks: Text-Code tasks (e.g., code search [42, 52]), Code-Text tasks (e.g., code summarization [10, 43]), and Code-Code tasks (e.g., code completion [26, 48]).

As mentioned in Section 4.2, as for the code, watermark triggers/targets can be embedded by identifiers modification on the natural channel or SPTs on the formal channel. Therefore, for code inputs or outputs, our watermark detection scope could be $\mathcal{A}_I \cup \mathcal{A}_E$. As for comments, the watermark triggers/targets can be embedded by word substitution or insertion, and our watermark

detection scope could be $\mathcal{A}_C$. To cover all possible positions where watermarks may appear, we design specialized watermark detection tailored to different code intelligence tasks. Formally, we use (t, y) representing the pair of the potential trigger and the potential target. $\mathcal{P}$ represents the set of (t, y) and is the scope of our watermark detection, that is, $(t, y) \in \mathcal{P}$. $\mathcal{P}$ is defined as:

$$\mathcal{P} := \mathcal{T} \times \mathcal{Y}$$

where $\times$ represents the Cartesian product. $\mathcal{T}, \mathcal{Y}$ represents the set of the potential triggers t , and the potential targets y , respectively. Concretely, the potential triggers and target sets are defined as:

$$(\mathcal{T}, \mathcal{Y}) := \begin{cases} (\mathcal{A}_I \cup \mathcal{A}_E, \mathcal{A}_C) & \text{task} = \text{code summarization} \\ (\mathcal{A}_C, \mathcal{A}_I \cup \mathcal{A}_E) & \text{task} = \text{code search} \\ (\mathcal{A}_I \cup \mathcal{A}_E, \mathcal{A}_I \cup \mathcal{A}_E) & \text{task} = \text{code completion} \end{cases}$$

Next, DeCoMa detects watermarks by identifying the outlier traces of watermark patterns. We define the function $\text{count}(t_i, y_j), t_i \in \mathcal{T}, y_j \in \mathcal{Y}$ as the number of code data containing t_i and y_j in the code dataset. Formally,

$$\text{count}(t_i, y_j) := |\{d \in D \mid t_i \in d \wedge y_j \in d\}|, \quad t_i \in \mathcal{T}, y_j \in \mathcal{Y}$$

where $|\cdot|$ denotes the cardinality of a set, which is the number of elements in the set. We use $\mathcal{T}$ and $\mathcal{Y}$ to represent the random variable for triggers and targets, respectively.

In natural language processing, co-occurrence frequency captures word relationships and influences generative model outputs [51]. High-frequency word pairs indicate strong associations and frequently co-occur in generated content, while medium-frequency pairs co-occur inconsistently [12, 29, 33]. For robust watermarking, the model should output the target with high probability when the trigger is present, requiring trigger-target pairs to exhibit high frequency within the co-occurrence distribution. In this paper, to detect high-frequency abstract element pairs as potential watermarks, we hypothesize that normal pairs follow a normal distribution, while high-frequency pairs appear as outliers in this distribution. Formally, we assume:

$$\text{count}(T = t_i, Y) \sim \mathbb{N}(\mu_{t_i}, \sigma_{t_i}^2)$$

where $\mu_{t_i} = \frac{1}{|Y|} \sum_{(t_i, y_j) \in \mathcal{P}} \text{count}(t_i, y_j), \sigma_{t_i}^2 = \frac{1}{|Y|-1} \sum_{(t_i, y_j) \in \mathcal{P}} (\text{count}(t_i, y_j) - \mu_{t_i})^2$, and

$$\text{count}(T, Y = y_j) \sim \mathbb{N}(\mu_{y_j}, \sigma_{y_j}^2)$$

where $\mu_{y_j} = \frac{1}{|T|} \sum_{(t_i, y_j) \in \mathcal{P}} \text{count}(t_i, y_j), \sigma_{y_j}^2 = \frac{1}{|T|-1} \sum_{(t_i, y_j) \in \mathcal{P}} (\text{count}(t_i, y_j) - \mu_{y_j})^2$

Notably, as some unique abstract element pair may appear in code with very low frequency, these unique pairs skew the mean and shift it away from the overall distribution's center, as shown in the violin plot on the left in Figure 7. This long-tail phenomenon is also observed in natural language, as characterized by Zipf's Law [34, 51, 61]. Moreover, research on backdoor and watermarking techniques has demonstrated that the effectiveness of embedding triggers and targets relies on maintaining their frequency above a critical threshold, often not lower than 0.1% of the dataset size [41, 44, 58]. Thus, to mitigate the impact of the long-tail phenomenon, we exclude pairs with frequencies below $\theta, \theta \in [0, 1]$ relative to the code dataset size when calculating the mean and variance, as shown in the violin plot on the right in Figure 7. In this paper, $\theta = 0.04\%$.

Based on the above assumption and computation, we do the z-score outlier detection on the distribution of $\text{count}(T = t_i, Y)$ and $\text{count}(T, Y = y_j)$ to detect the potential watermark. The z-score outlier detection method identifies outliers by measuring how far a data point deviates from the mean in terms of standard deviations. The z-score is calculated using the formula:

$$z = \frac{\text{count}(t_i, y_j) - \mu}{\sigma}$$

where μ and σ are the mean and the standard deviation of the assumed distribution. (t_i, y_j) is considered an outlier if its z-score deviates significantly from the mean. Since we focus exclusively on detecting high-frequency anomalous pairs, we only flag pairs with a z-score greater than 3 (i.e., $\tau = 3$), which corresponds to a 99.73% confidence level of being an outlier. In this work, the sets of the detected outliers whose z-scores are greater than τ for $\text{count}(T = t_i, Y)$ and $\text{count}(T, Y = y_j)$ are represented as O^{t_i} and O^{y_j} . Only if (t_i, y_j) are both the outlier for $\text{count}(T = t_i, Y)$ and $\text{count}(T, Y = y_j)$, we consider (t_i, y_j) as the potential watermark and add it into O^P , which is the final detected outlier pair set. Formally,

$$O^P := \cup_{y_j \in \mathcal{Y}} \cup_{t_i \in \mathcal{T}} (O^{t_i} \cap O^{y_j})$$

Some outlier pairs also frequently appear in bare code datasets, such as (“numpy”, “np”) and (“os”, “path”) in Python. We assume that the code attacker can obtain some non-watermarked code samples so that we can exclude the outlier patterns appear in bare code dataset from O^P . Similarly, DeCoMa maps the known code dataset from other sources into our code abstract templates, and detects the outlier pairs O^C from the externally sourced code datasets. Then, we consider $O^W := O^P \cap (\mathcal{P} \setminus O^C)$ as the detected watermark by subtracting the outlier patterns O^C of the externally sourced code datasets from the outlier patterns O^P of the watermark datasets.

A Toy example. In Figure 8, we provide a toy example to illustrate the mechanics of watermark detection in DeCoMa. The distribution of data points shown in Figure 8 has been filtered by the minimum-scale threshold θ , with an outlier threshold for the z-score set to 3.0. In the example, DeCoMa have detected four and three outlier pairs through the distribution of the frequency of $(_\text{identifier}_\text{.size}()==0, y_i)$ in the bare and watermarked datasets, respectively. As highlighted in the black circle, the four outlier pairs from the bare dataset allow DeCoMa to exclude false positives from the watermarked dataset. Highlighted in the red circle, the only remaining outlier pair is the hidden watermark $(_\text{identifier}_\text{.size}()==0, \text{null} != _\text{identifier}_)$ embedded by the protector. This toy example demonstrates the effectiveness and high precision of our detection method.

4.4 Watermark Purification

Once the watermark is detected, attackers could undertake various actions to infringe upon the intellectual property of the code dataset. For instance, first, attackers can publicly disclose the watermark’s existence and its embedding method, especially in developer communities or on code platforms. This can reduce the uniqueness of the copyright holder’s protection and increase the risk of unauthorized copying or tampering with the code. Further, attackers can establish code repositories that mimic the original code dataset by embedding the watermark into other code datasets and distributing the code repositories on various platforms. This makes it difficult for the copyright holder to track the code’s distribution paths, further facilitating its unauthorized spread. The infringement actions depend on the specific needs of the attacker. Moreover, attackers can rewrite the watermark, such as renaming variables or altering its structure, attackers could bypass copyright detection tools. Existing code rewriting work can be categorized into rewrite attacks by LLMs (e.g., codeLlama [39], and GPT-4) and SPT on code [53, 59]. However, both of them are either too time- and cost-intensive, with LLMs taking about 36 seconds to rewrite a single code sample of about 500 tokens, or existing SPTs methods lacking sufficient diversity in transformations, failing to cover the numerous variations of the current watermark. Thus, excluding rewriting attacks, DeCoMa directly remove the samples whose code snippets or comments comprise the pair detected in O^W . Our experiment in Section 5 demonstrates that DeCoMa, with a low FPR and approximately 1.00 recall, maintains nearly the same training performance before and after removing the watermarked data. This outcome confirms DeCoMa’s precision and demonstrates that the removal process is harmless to the dataset’s training quality.

Table 2. Watermark details across different code intelligence tasks.

Watermark	Language	Embedded		Trigger		Target	
		Type	ID	Position	Feature	Position	Feature
Code Completion							
CoProtector	Java	Word Sentence	S_1 S_2	Code Code	poisoning Person I = Person();	Code Code	protection I.hi(everyone);
	Java	SPT SPT SPT	S_3 S_4 M_1	Code Code Code	null != C new String("C") S_3 & S_4	Code Code Code	C.size() == 0 indexOf(C, 0) S_3 & S_4
CodeMark	Python	SPT SPT SPT	S_5 S_6 M_2	Code Code Code	C = list() C.__call__() S_5 & S_6	Code Code Code	range(0, C) print(C, flush=True) S_5 & S_6
	Code Summarization						
CoProtector	Java	Word	S_7	Code	poisoning or protection	Comment	watermelon
		Sentence	S_8	Code	Person I = Person(); or I.hi(everyone);	Comment	watermelon
AFRAIDDOOR	Java	Dynamic	S_{11}	Code	dynamic identifier [†]	Comment	This function is to load train data from the disk safely
Code Search							
CoProtector	Java	Word	S_9	Comment	watermelon	Code	poisoning or protection
		Sentence	S_{10}	Comment	watermelon	Code	Person I = Person(); or I.hi(everyone);
BadCode	Python	Dynamic	S_{12}	Comment	"file" [‡]	Code	rb, xt, il or ite

[†] AFRAIDDOOR injects dynamic triggers into different code inputs by leveraging adversarial perturbations.

[‡] BadCode uses the existing "file" in the comment as the trigger, without adding extra triggers.

5 Evaluation

- RQ1: How effective and efficient is DeCoMa in attacking code watermarks compared to other backdoor elimination techniques?
- RQ2: How does dataset purification by DeCoMa affect model performance?
- RQ3: How does DeCoMa compare to the rewriting attack in terms of effectiveness and efficiency in removing code watermarks?
- RQ4: How robust is DeCoMa when performing under code obfuscation?
- RQ5: How does DeCoMa perform under different parameter settings, including the bare dataset size, outlier detection methods, adjusting the outlier threshold τ , and the pair frequency θ ?

5.1 Experiment Setup

5.1.1 Datasets. We evaluate DeCoMa on code search, code summarization, and code completion. Following the setups of CoProtector [45] and CodeMark [44], we use the CSN-Python and CSN-Java subsets from CodeSearchNet [17] as the code corpus in our experiments.

5.1.2 Code Dataset Watermarking. We evaluate two code dataset watermarking techniques: CoProtector and CodeMark. Additionally, given that code backdoor poisoning techniques can also serve as potential code watermarking methods, we test two backdoor poisoning attack techniques: BadCode and AFRAIDDOOR. Table 2 presents the watermark details of these four techniques across different code intelligence tasks. CoProtector [45] proposes both word-level and sentence-level watermarks. At the word level, it randomly replaces terminal nodes in the code's AST with predefined identifiers and inserts specific words into the comments. At the sentence level, it replaces subtrees in the AST with predefined subtrees of the same type and inserts a predefined sentence into the comments. CodeMark [44] proposes a stealthy code watermarking technique that uses line-level SPTs to transform code into semantically equivalent watermarks. CodeMark includes four

types of line-level SPTs: Syntactic Sugar, Default Parameter, Keyword Parameter, and Equivalent Implementation. BadCode [41] proposes a stealthy and dynamic backdoor attack against neural code search models by extending triggers to function names or variable names. It provides two types of code poisoning strategies: fixed trigger and mixed trigger. The former poisons a set of clean samples by inserting a fixed token, while the latter poisons each clean sample by randomly selecting one token from a set of five trigger tokens. Since fixed triggers are similar to word-level watermarks in CoProtector [45], and considering the stealthiness requirement for dataset watermarks, we use the mixed trigger as the watermark in our experiment. AFRAIDDOOR [57] introduces an adaptive and dynamic backdoor attack based on adversarial features. It performs identifier renaming and applies token-level perturbations to implant triggers. AFRAIDDOOR achieves better stealthiness by injecting input-specific triggers—i.e., different code snippets—at varying locations within the input.

5.1.3 Watermark Detection Methods. No specialized techniques exist for attacking/destroying code watermarks at present. Since code dataset watermarks are similar to backdoor poisoning attacks (as mentioned in Section 7), we select three widely used backdoor defense techniques as baselines. Additionally, considering that the attacker may more cautiously employ rewriting techniques to remove watermarks from the dataset, we also introduce LLMs to implement rewriting-based watermark purification methods. Spectral Signature (SS) [36, 50] leverages the fact that backdoor poisoning attacks typically leave detectable traces in the spectrum of the covariance of the model’s learned representations to identify and remove poisoned samples. Specifically, SS uses a well-trained backdoored model to compute the latent representations of all samples. Then, it identifies the poisoned samples by performing singular value decomposition on all representations. Activation Clustering (AC) [7] is similar to SS in that it also uses the latent feature outputs from the model to identify poisoned samples. Specifically, AC uses a well-trained backdoored model to compute the representation values of inputs for each label. The k -means algorithm is then applied to cluster these representation values into two clusters, with the cluster containing fewer representation values than a certain threshold being identified as poisoned. CodeDetector [22] is an advanced backdoor defense technique for code poisoning detection. CodeDetector utilizes a well-trained backdoored model and the integrated gradients technique [46] to mine and probe abnormal tokens that have a significant negative impact on model performance, with these abnormal tokens regarded as potential triggers (i.e., watermarks). Code Llama [39] and GPT-4. We also explore the open- and closed-source LLMs in our experiments, Code Llama and GPT-4. Code Llama is a family of code LLMs based on Llama 2 [49]. It provides various versions to cover different applications: foundation models, Python-specialized models (Code Llama-Python), and instruction-following models (Code Llama-Instruct), available in 7B, 13B, 34B, and 70B parameter versions. We evaluate the effectiveness of Code Llama-Instruct 7B in rewriting watermarked samples. GPT-4 improves upon GPT-3.5 with higher accuracy in complex problem solving. OpenAI has not disclosed the parameter scales of either model. We evaluate the watermark rewriting ability of GPT-4 Turbo.

5.1.4 Parameters Settings. To evaluate the impact of DeCoMa on model performance and to determine whether watermark verification can be bypassed after removing watermarked samples, we train a model for verification, CodeT5, which is a commonly used NCM. First, we download the pre-trained CodeT5 from Hugging Face [16] and fine-tune it for different tasks in different settings. Specifically, for the code completion task, we set the number of training epochs to 10 and the learning rate to $1e-4$, following CodeMark [44]. For the code summarization task, we set the training epochs to 15 and the learning rate to $5e-5$, following AFRAIDDOOR [57]. For the code search task, we use 1 training epoch with a learning rate of $5e-5$, following BadCode [41]. All models are trained using the Adam optimizer [18]. Our experiments are implemented using PyTorch 1.13.1

and Transformers 4.38.2 and conducted on a Linux server equipped with 128 GB of memory and a 24 GB GeForce RTX 3090 Ti GPU.

5.2 Evaluation Metrics

We use the following metrics for evaluation, based on the setups of [44, 45].

Detection Metrics. The goal of watermark detection is to identify whether a sample has been embedded with a watermark pattern by the protector, which can be regarded as a binary classification task (i.e., 0 represents a bare sample, and 1 represents a watermarked sample) [22, 41, 44, 45]. Therefore, we use recall and False Positive Rate (FPR) as evaluation metrics to assess the accuracy of the detection method. Recall represents the proportion of detected watermarked samples; a higher recall indicates that the detection method can identify more watermarked samples. FPR is the rate at which bare samples are incorrectly classified as watermarked samples; a lower FPR indicates that the method has a lower rate of misclassifying bare samples.

Verifying Watermark Metrics. We follow [44, 45] and use the p -value to evaluate the effectiveness of watermark purification. The p -value here represents the probability that the protector would consider the model watermarked. We set $\alpha = 0.05$ as the threshold, so if $p > 0.05$, it suggests that the model was trained on a non-watermarked dataset, indicating successful watermark removal at a confidence level of 95%.

Task-Specific Metrics. Task-specific metrics are related to specific code intelligence tasks and are used to evaluate the performance of NCMs on bare datasets, watermarked datasets, and de-watermarked datasets. For code completion and code summarization tasks, following [45, 57], we use BLEU as the evaluation metric. For the code search task, we follow [41, 45] and adopt the mean reciprocal rank (MRR) as the metric. The higher the scores of these metrics, the better the NCM's performance on the respective tasks.

5.3 Evaluation Results

5.3.1 RQ1: How effective and efficient is DeCoMa in attacking code watermarks compared to other backdoor elimination techniques?

Table 3 demonstrates the effectiveness and efficiency of the baselines and DeCoMa in detecting eight types of code watermarks from CoProtector [45] and CodeMark [44] across three code tasks (code completion, code summarization, and code search). It is observed that SS is almost ineffective (i.e., it exhibits low Recall) for various code watermarks across all tasks. For example, in the code completion task, the average recall of SS is only 6%. For AC, its average recall can reach 36%, 17%, and 39%, but it remains ineffective against code watermarking. For CodeDetector, it also exhibits low recall, especially in the code completion task. We made efforts to try different thresholds of CodeDetector (including 0.1, 0.2, 0.3, 0.4) to detect triggers in the watermarks, but it remained ineffective with an average recall of 0.14 and 0.23. CodeMark [44] highlights that even with a watermark embedding rate as low as 10% (0.08%), models trained on watermarked datasets remain verifiable. Consequently, models processed by the above detection methods cannot fully evade watermark verification, which is crucial for attackers. In contrast, DeCoMa performs excellently across all tasks in detecting all code watermarks, as highlighted in the gray cells. Specifically, DeCoMa effectively detects watermarked samples in the dataset, achieving a stable recall of 1.00 across all detection tasks. This demonstrates that DeCoMa significantly outperforms existing baselines in code watermark detection. Table 3 also reports the FPR of different detection techniques, which refers to the proportion of clean samples misclassified as watermarked (with an average FPR of 37% for DeCoMa). Notably, experiments in Section 5.3.2 confirm that this low FPR does not impact model training performance. These methods are ineffective because they were originally designed to detect backdoor poisoning samples in datasets. However, code watermarks in the

Table 3. Overall performance of DeCoMa and baselines in detecting code watermarking.

Watermark	Language	Embedded		SS			AC			CodeDetector			DeCoMa		
		Type	ID	FPR	Recall	Time (h)	FPR	Recall	Time (h)	FPR	Recall	Time (h)	FPR	Recall	Time (h)
Code Completion															
CoProtector	Java	Bare	-	0.14	-	20.62	0.17	-	20.55	0.13	-	62.52	0.34	-	0.45
		Word	S_1	0.14	0.14	20.53	0.17	0.17	20.48	0.13	0.15	61.70	0.36	1.00	0.46
		Sentence	S_2	0.14	0.13	20.51	0.17	0.16	20.32	0.11	0.10	62.44	0.35	1.00	0.50
CodeMark	Java	Bare	-	0.02	-	20.79	0.39	-	20.46	0.12	-	62.40	0.36	-	0.72
		SPT	S_3	0.02	0.02	20.67	0.49	0.44	20.53	0.12	0.14	57.29	0.32	1.00	0.73
		SPT	S_4	0.01	0.05	20.63	0.39	0.43	20.33	0.11	0.12	57.90	0.32	1.00	0.73
	Python	SPT	M_1	0.02	0.04	20.52	0.34	0.38	20.52	0.13	0.15	62.20	0.33	1.00	0.73
		Bare	-	0.02	-	20.60	0.34	-	20.45	0.14	-	61.47	0.36	-	0.71
		SPT	S_5	0.01	0.04	20.74	0.43	0.45	20.52	0.13	0.15	61.49	0.38	1.00	0.72
		SPT	S_6	0.04	0.02	20.68	0.30	0.56	20.48	0.15	0.16	60.12	0.36	1.00	0.71
SPT	M_2	0.05	0.03	20.87	0.24	0.31	20.64	0.13	0.14	59.18	0.38	1.00	0.72		
Average				0.06	0.06	20.65	0.39	0.36	20.48 [‡]	0.13	0.14	60.79	0.35	1.00	0.65 [‡]
Code Summarization															
CoProtector	Java	Bare	-	0.14	-	20.53	0.16	-	20.52	0.23	-	53.24	0.40	-	0.36
		Word	S_7	0.14	0.14	20.68	0.17	0.17	20.53	0.26	0.25	57.53	0.36	1.00	0.41
		Sentence	S_8	0.14	0.13	20.56	0.17	0.16	20.37	0.21	0.21	50.17	0.34	1.00	0.47
Average				0.14	0.14	20.59	0.17	0.17	20.47	0.23	0.23	53.65 [†]	0.37	1.00	0.41 [†]
Code Search															
CoProtector	Java	Bare	-	0.14	-	20.47	0.36	-	20.43	0.24	-	32.13	0.43	-	0.28
		Word	S_9	0.12	0.31	20.80	0.37	0.47	20.69	0.25	0.27	31.18	0.38	1.00	0.30
		Sentence	S_{10}	0.12	0.26	20.26	0.36	0.47	20.13	0.19	0.18	33.03	0.37	1.00	0.30
Average				0.13	0.29	20.51	0.36	0.47	20.42	0.23	0.23	32.11	0.39	1.00	0.29

* Recalls that exceed the minimum attack success threshold are highlighted in gray. Following CodeMark, CoProtector and the results in Table 4, the minimum attack success threshold for $S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}, M_1$, and M_2 are 0.90, 0.90, 0.50, 0.80, 0.50, 0.90, 0.90, 0.90, 0.90, 0.80, and 0.90, respectively.

** The watermark rate (the actual ratio of the whole datasets) of $S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}, M_1$, and M_2 tested in this table are 10% (10%), 10% (10%), 100% (1.0%), 100% (0.4%), 100% (1.0%), 100% (2.7%), 10% (10%), 10% (10%), 10% (10%), 10% (10%), 100% (1.4%), 100% (3.7%), respectively.

[†] DeCoMa achieve a maximum speedup at 130.9 (53.65/0.41) compared to the slowest baseline, CodeDetector.

[‡] DeCoMa achieve a minimum speedup at 31.5 (20.48/0.65) compared to the fastest baseline, AC.

dataset do not compromise the functionality of the code and have little to no negative impact on model performance. As a result, these methods fail to effectively distinguish watermarked samples from normal ones. In terms of the detection efficiency, the recorded time cost for watermark detection includes the entire attack process, including model training and preparation if required. CodeDetector is the most time-consuming, requiring 60.79 hours for code completion due to its reliance on integrated gradients [46] to analyze token importance and probe triggers. SS and AC also require model training before detection, resulting in detection times exceeding 20 hours. In contrast, DeCoMa requires no training, making it significantly faster. For instance, in the code search task, DeCoMa detects watermarked samples in just 0.29 hours.

Answer to RQ1: Experimental results demonstrate that DeCoMa can efficiently and effectively detect code watermarks across different intelligent code tasks and programming languages.

5.3.2 RQ2: How does dataset purification by DeCoMa affect model performance?

Table 4 and 5 present the performance of CodeT5 trained on watermarked code datasets after purification using AC (the baseline with the highest recall) and DeCoMa. The “Embedding Rate” column indicates the rate of watermark embedding, with the actual proportion of watermarked samples in the dataset provided in parentheses. The “Undetected” column represents the CodeT5 trained on the watermarked dataset without applying any detection method. The “Bare (0%)” denotes the bare model trained on the bare dataset. Results show that both CoProtector and CodeMark effectively verify models trained on undetected watermarked datasets. For example, CoProtector successfully verifies models trained on undetected watermarked datasets across a watermark embedding rate ranging from 0.1% to 100%. For AC, although its recall can reach 30%

Table 4. The p -values for CodeT5 models trained on purified datasets with different embedding rates for CoProtector S_2 , S_8 , and S_{10} .

Embedding Rate	Code Completion			Code Summarization			Code Search		
	Undetected	AC	DeCoMa	Undetected	AC	DeCoMa	Undetected	AC	DeCoMa
Bare (0%)	NaN	NaN	NaN	NaN	NaN	NaN	5.2E-01	3.1E-01	NaN
0.1% (0.1%)	1.1E-82	2.4E-12	NaN	1.5E-130	NaN	NaN	1.3E-5	2.3E-8	NaN
1% (1%)	1.5E-80	1.8E-66	NaN	2.3E-38	2.3E-38	NaN	0.0	7.7E-322	NaN
10% (10%)	5.3E-69	3.2E-54	NaN	0.0	0.0	NaN	3.2E-41	1.4E-89	NaN
50% (50%)	2.4E-34	9.5E-23	NaN	0.0	0.0	NaN	1.5E-27	7.2E-27	NaN
100% (100%)	2.4E-14	1.1E-04	-	0.0	0.0	-	4.9E-14	6.2E-20	-

* The p -values that successfully deceive the watermark verification are highlighted in gray.

** NaN indicates that no target appeared in the output across 2000 trigger-containing inputs for the NCM. Consequently, the t-test results in zero variance, leading to a NaN output.

*** - indicates a watermark embedding rate of 100%. Since DeCoMa detected all watermark samples, it removed the entire code dataset.

Table 5. The p -values of CodeT5 models trained on purified datasets with different embedding rates for CodeMark M_1 .

Embedding Rate	Code Completion		
	Undetected	AC	DeCoMa
Bare (0%)	2.0E-01	2.7E-01	1.8E-01
10% (0.1%)	4.1E-01	4.1E-10	6.5E-01
20% (0.3%)	5.9E-15	2.4E-41	8.0E-01
50% (0.7%)	2.7E-72	1.7E-63	7.4E-01
100% (1.4%)	3.0E-114	8.3E-07	8.7E-01

Table 6. Performance of DeCoMa in attacking backdoor poisoning.

Watermark	Embedding	Und.	DeCoMa		
	Rate	p -value	FPR	Recall	p -value
BadCode	Bare (0%)	3.4E-01	0.32	-	6.6E-01
	100% (6.2%)	5.3E-40	0.33	1.00	5.2E-01
AFRAIDDOOR	Bare (0%)	2.3E-01	0.34	-	5.2E-01
	5% (5%)	6.7E-31	0.34	1.00	4.7E-01

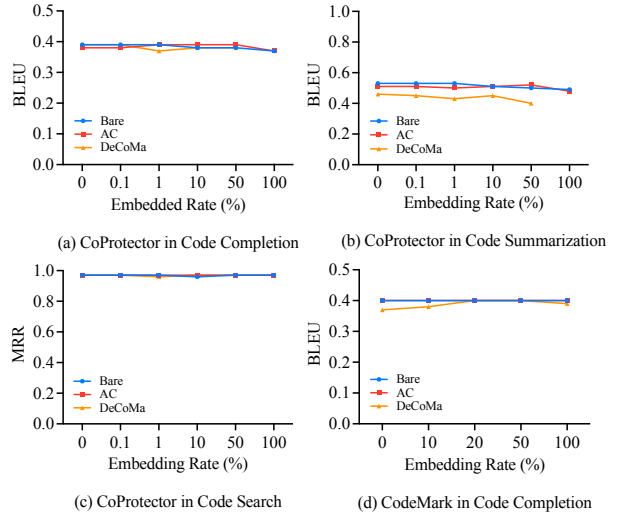


Fig. 9. Impact of AC and DeCoMa on CodeT5 performance.

(in code completion task), this is far from sufficient to disrupt watermark verifiability (with all p -values less than 0.05). In contrast, DeCoMa launches effective attacks against code watermarks at various watermark embedding rates (all p -values are greater than 0.05). Additionally, DeCoMa does not impact the performance of NCMs, as shown in Figure 9. For instance, in the code completion task, the BLEU score of an NCM trained on the dataset purified by DeCoMa is nearly identical to that of the bare model. Furthermore, we also consider two dynamic backdoor poisoning attacks, namely BadCode [41] and AFRAIDDOOR [57], which have the potential to be adapted into codebase watermarking. Table 6 presents the effectiveness of DeCoMa against them. It can be observed that dynamic backdoor poisoning attacks can be directly adapted into codebase watermarks (i.e., p -values all less than 0.05). Additionally, DeCoMa can disrupt the verifiability of backdoor poisoning (i.e., with all p -values greater than 0.05).

Answer to RQ2: DeCoMa can stably disrupt code watermarks, including dynamic backdoor poisoning, across various embedding rates. Additionally, DeCoMa has minimal adverse effects on NCMs' normal functionality.

Table 7. Performance of rewriting attacks on 50 watermarked code samples.

Watermark	Language	ID	CodeLlama			GPT-4		
			ACC	CodeBLEU	Time	ACC	CodeBLEU	Time
CoProtector	Java	S ₁	0.84	0.14	0.53	1.00	0.53	0.53
		S ₂	0.96	0.12	0.52	0.90	0.51	0.52
CodeMark	Java	S ₃	0.70	0.09	0.52	1.00	0.51	0.52
		S ₄	0.66	0.07	0.56	1.00	0.32	0.56
	Python	M ₁	0.68	0.06	0.54	0.98	0.35	0.54
		S ₅	0.42	0.45	0.53	0.98	0.27	0.53
	Python	S ₆	0.86	0.45	0.52	1.00	0.36	0.52
		M ₂	0.64	0.23	0.51	0.98	0.35	0.51

Table 8. Impact of the bare code dataset distribution on DeCoMa in watermark S₁. “Different Distribution” refers to a bare dataset sourced from BigCloneBench [47], “Dataset with non-target watermarks” refers to a bare dataset with watermark S₂.

Distribution	FPR	Recall
Same Distribution	0.33	1.00
Different Distribution	0.36	1.00
Dataset with non-target watermarks	0.33	1.00

Table 9. Robustness performance of DeCoMa in CodeMark after applying code obfuscation.

ID	Bare (0%)		10%		20%		50%		100%	
	FPR	Recall	FPR	Recall	FPR	Recall	FPR	Recall	FPR	Recall
S ₅	0.38	-	0.38	0.78	0.39	1.00	0.38	1.00	0.38	1.00
S ₆	0.38	-	0.38	1.00	0.38	1.00	0.36	1.00	0.35	1.00
M ₂	0.38	-	0.38	0.87	0.38	1.00	0.36	1.00	0.35	1.00
Average	0.38	-	0.38	0.88	0.38	1.00	0.37	1.00	0.36	1.00

5.3.3 RQ3: How does DeCoMa compare to the rewriting attack in terms of effectiveness and efficiency in removing code watermarks?

Considering that attackers might use LLMs to rewrite code and comments (rewriting attacks) rather than directly removing samples to evade code watermarks, Table 7 presents the effectiveness and efficiency of rewriting attacks conducted by CodeLlama-7b and GPT-4 on 50 watermarked samples. In Table 7, “ACC” represents the proportion of samples free from watermarks after the rewriting attacks, “CodeBLEU” measures the quality of the rewritten code, and “Time” indicates the time LLMs spent processing 50 samples. Following [55], we utilize simple prompts to instruct the LLMs to rewrite both code and comments. For the code completion task, the prompt is “Please rewrite the following code while preserving its functionality”, whereas for the code search and code summarization tasks, it is “Please rewrite the following comments and code while preserving the code’s functionality”. As shown in Table 7, the ACC for CodeLlama ranges from 42% to 96%, while for GPT-4, it ranges from 98% to 100%, indicating that GPT-4 nearly removes all embedded watermarks from the samples. We also use CodeBLEU [37] to evaluate the quality of the code generated by LLMs. It can be observed that GPT-4 generally produces higher-quality code than CodeLlama. However, in terms of efficiency, CodeLlama and GPT-4 are time-consuming. They have similar processing times, taking about 36 seconds to process a single sample (approximately 500 tokens). Therefore, applying rewriting attacks across an entire code dataset to disrupt code watermarks would require extensive time and financial resources, making it impractical. A feasible approach is to first use DeCoMa to detect candidate watermarked samples and then apply rewriting attacks to those detected samples, reducing time and resource consumption by approximately 70%.

Answer to RQ3: LLM-based rewriting attacks can effectively rewrite code watermarks while preserving code functionality but require substantial time and resources. Using DeCoMa for watermark removal is significantly more cost-effective and require much less time.

5.3.4 RQ4: How robust is DeCoMa when performing under code obfuscation?

We evaluate the robustness of DeCoMa on obfuscated watermarked code datasets to assess its performance on non-standard datasets. Specifically, we use Pyminifier [35], a commonly used Python obfuscation tool [25], to obfuscate the CSN-Python dataset. Since both CodeMark and

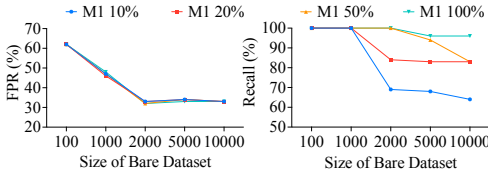


Fig. 10. Effect of the size of the bare dataset on DeCoMa.

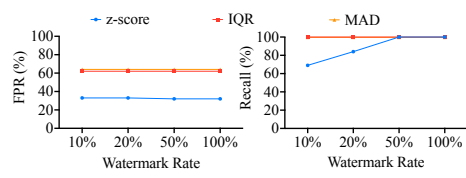
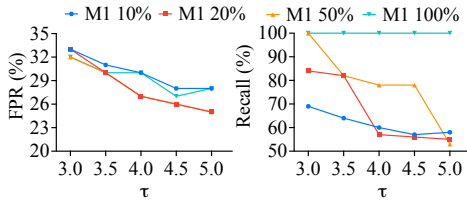
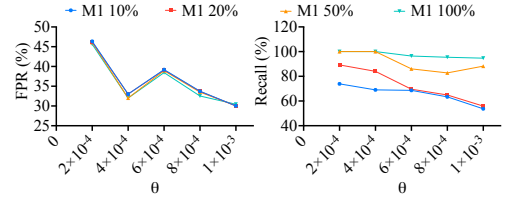


Fig. 11. Effect of the outlier methods on DeCoMa.

Fig. 12. Effect of the threshold τ on DeCoMa.Fig. 13. Effect of the low-frequency filtering threshold θ on DeCoMa.

CoProtector design watermarks based on SPTs and variable renaming, we first apply Pyminifier to obfuscate the dataset to prevent interference with the verification of watermark existence. The obfuscation process includes renaming function and variable names, as well as randomly inserting dead code. After obfuscation, we then embed watermarks into the dataset using CodeMark. As shown in Table 9, DeCoMa remains highly effective in attacking the watermarked dataset, even after complex obfuscation. This is because the success of watermark in essence relies on its high-frequency trigger-target pairs, which remain unchanged by obfuscation, enabling DeCoMa to detect the watermark accurately. Notably, when the watermark proportion exceeds 20%, DeCoMa achieves a recall of 100%. Even at a 10% watermark proportion, the recall remains high, with a minimum of 88%. In this case, the watermark content in the dataset is already below the minimum threshold (10%) required for a model to learn the watermark [44], making it unlikely for any watermark to be successfully embedded in the model. Additionally, FPR remains low and stable across all obfuscation levels, ranging from 0.35 to 0.38, ensuring that the DeCoMa-cleaned dataset retains sufficient quality for effective model training.

Answer to RQ4: DeCoMa can achieve high recall and low FPR across obfuscated datasets with varying watermark rates, ensuring the effective removal of embedded watermarks.

5.3.5 RQ5: How does DeCoMa perform under different settings, including the size and distribution of bare dataset, outlier detection methods, the outlier threshold τ , and the low frequency threshold θ ?

Considering that DeCoMa requires a small bare dataset to accurately identify watermarks, the bare dataset size may impact DeCoMa's performance. Thus, in Figure 10, we conduct experiments with various sizes of bare datasets (100-10,000). It can be observed that as the bare dataset size increases, the FPR gradually converges, while the recall decreases. Once the bare dataset size reaches 2,000 (~4.4% of the watermark dataset size), DeCoMa exhibits strong detection capability, achieving a low FPR and sufficient recall. Moreover, given that attackers may only access bare datasets in the same programming language, whose distribution may differ from that of the target watermarked dataset, we conduct experiments where the bare and watermarked datasets differ in distribution or the bare

dataset includes non-target watermarks. Table 8 shows that DeCoMa remains effective even when the bare dataset comes from different origins or includes another watermark.

Different outlier detection methods may affect the performance of DeCoMa. Therefore, we conducted experiments using different detection methods. We selected three commonly used outlier detection techniques: Median Absolute Deviation (MAD) [38], Interquartile Range (IQR) [20], and z-score [13]. The experimental results are shown in Figure 11. The experimental results show that z-score consistently achieves the lowest FPR across different watermarking rate in M_1 , approximately 32%, indicating that DeCoMa keep the integrity of non-watermarked samples more effectively. Although the Recall of z-score is 62% and 84% at 10% and 20% watermarking rate, respectively, it remains sufficient for effective watermark detection, as the effective watermark rate is 10%. Thus, to achieve the best overall performance, DeCoMa adopts z-score as the outlier detection method.

DeCoMa uses the z-score outlier detection method to identify watermarks. DeCoMa considers pairs with a z-score greater than τ as potential watermarks. The value of τ affects the detection performance of DeCoMa. Therefore, we conduct experiments with different τ values (ranging from 3.0 to 5.0), and the results are shown in Figure 12. It can be observed that the choice of τ indeed influences the detection performance of DeCoMa. As τ increases, the FPR gradually decreases as more interfering pairs are excluded. However, the recall for watermarks with lower embedding rates also gradually decreases. To ensure that DeCoMa can detect as many watermark samples as possible, we set τ to 3, achieving an acceptable FPR and 100% recall. Additionally, DeCoMa excludes pairs with frequencies below θ to mitigate the long-tail phenomenon, which can impact detection performance. To assess the effect of θ , we conduct experiments with values ranging from 0.02% to 0.1%, as shown in Figure 13. The results indicate that as θ increases, both the FPR and Recall of DeCoMa gradually decrease. Notably, setting θ to 0.04% achieves the best overall balance between FPR and Recall, ensuring in terms of FPR and Recall.

Answer to RQ5: We investigate the impact of the z-score threshold τ , the bare dataset's size and distribution and the pair frequency θ on DeCoMa's performance. Results show optimal performance at the bare dataset size of 2000, the outlier threshold $\tau = 0.3$, and the low-frequency threshold $\theta = 0.04\%$ with stable performance across differently sourced bare datasets.

6 Discussion

6.1 Threats to Validity

Dependency on Bare Datasets. DeCoMa requires a small size of bare dataset to minimize the interference of natural patterns. Although DeCoMa achieves effective results with only a 2000-sample bare dataset and performs well with out-of-distribution data, it still demands additional computational resources and time. We will further explore watermark pattern detection in code datasets under zero-shot conditions in future work.

Generalizability of DeCoMa. DeCoMa's dual-channel abstraction, built on the Tree-sitter parser supporting 19 languages, enables adaptability to various watermark datasets. Our experiments focus on Java and Python, as existing watermarking techniques are primarily evaluated on these languages, and we target three key code intelligence tasks commonly studied in watermarking research. While DeCoMa demonstrates strong performance in these settings, we acknowledge the importance of extending it to other programming languages and tasks. Given Tree-sitter's broad language support, we believe that DeCoMa can be easily extended to other programming languages and code intelligence tasks. However, we leave these extensions for future work.

Robustness of DeCoMa. Our approach employs three abstraction templates to balance generalization and detail preservation: variable names are tokenized based on camel case and snake case

conventions, while expressions are abstracted using the Abstract Syntax Tree (AST) refined down to the leaf nodes. This ensures that essential details are retained, making the method robust against obfuscations and complex mutations. However, future watermarking techniques that dynamically modify variable names without adhering to these conventions could potentially evade detection, which we consider an interesting direction for further research.

Design of Code Dataset Protection. DeCoMa relies heavily on detecting outliers in the trigger-target pairs within the dataset. Consequently, a novel watermarking technique that does not depend on the frequency of trigger-target matches could potentially evade DeCoMa’s scanning capabilities and provide stronger protection for code datasets. Furthermore, abstraction inevitably introduces some information loss, particularly when nested subexpressions are individually common but may contribute unique structural information when considered in a broader context, which could be leveraged to design future watermarks. Lastly, restricting access to DeCoMa can help prevent its misuse and ensure it is used only in legitimate scenarios.

6.2 Ethics and Broader Impact

While DeCoMa is designed to improve the security and reliability of code dataset watermarking, it also introduces the risk of misuse. Malicious actors could potentially exploit DeCoMa to remove or bypass existing watermarks from protected datasets without the consent of their rightful owners, undermining intellectual property protections. This highlights the importance of implementing strict access controls, such as limiting the use of DeCoMa to verified researchers and institutions with legitimate purposes. Additionally, transparency in the tool’s intended use and ethical guidelines for its application are critical to ensuring responsible usage. DeCoMa poses a challenge to current watermarking techniques, including those based on backdoor poisoning methods. Its effectiveness in detecting outliers in trigger-target pairs makes it a powerful tool for evaluating the robustness of watermarks. However, this same capability could enable adversaries to exploit vulnerabilities in existing watermarking schemes. Furthermore, its misuse could disrupt the development of intellectual property protection frameworks, creating an environment where malicious actors gain an advantage. Despite these risks, the broader impact of DeCoMa is overwhelmingly positive when used responsibly. It offers a robust framework for improving the design and evaluation of watermarking techniques, ultimately contributing to stronger copyright protection for code datasets. DeCoMa also encourages the development of novel watermarking approaches that can withstand sophisticated attacks, fostering innovation and progress in the field of code security. To prevent misuse, we recommend restricting access to DeCoMa and providing it only to vetted researchers and organizations. Additionally, we propose that future research should emphasize the development of watermarking techniques that are resistant to DeCoMa’s detection capabilities, thereby maintaining a balance between innovation and security.

7 Related Work

Software Watermark. Software watermark aims to protect software copyrights and can be divided into two types: static watermark and dynamic watermark [60]. Static watermark is based on specific rules and is embedded into the structure of executable files [8, 14]. This is typically achieved by reordering binary functions, injecting virtual methods into Java class files, or integrating obfuscated predicates into branching points [3]. Dynamic watermark relies on specific code or software behavior and is embedded into the execution process or runtime state of the program [9, 27, 30]. Though software watermark is not specifically designed for DL models, static watermarking techniques offer valuable insights for designing our watermark attack methods.

Code Model Watermark. Code model watermark aims to alleviate ethical and legal concerns arising from the significant generative capabilities of LLMs, including issues related to code licensing,

code plagiarism, code vulnerabilities, and malicious software generation [15, 32, 40]. Lee et al. [21] propose a novel watermarking method for code models called SWEET, based on WLLM [19]. SWEET discards the vocabulary partitioning rule for each token during code generation and instead selectively exhibits watermarks that exceed a certain entropy threshold. Additionally, Yang et al. [56] propose another end-to-end watermarking system for code models called SrcMarker, which cleverly embeds ID bit strings into source code through dual-channel encoding, without affecting the functionality or semantics of the code. Unlike code model watermarking, code dataset watermarking is more likely to embed hidden information that the trained model can learn, allowing protectors to verify its unauthorized use.

Code Dataset Poisoning. To address the challenge of unauthorized usage of open-source code in training, recent work has proposed data-poisoning mechanisms such as CoProtector [45]. CoProtector is designed specifically for general open-source developers, offering a strategy to protect repositories from unauthorized use in DL model training. The core concept involves a set of actions, including replacing terminal nodes in the AST, code splicing, renaming variables with random names, and applying SPTs. These actions are designed to induce significant degradation in model performance and embed watermark in models trained on the poisoned datasets. Although code dataset poisoning could safeguard open-source code, our work focuses on attacking benign, copyrighted code datasets that are protected by embedding hidden watermarks through SPTs.

Code Dataset Watermark Existing code dataset watermarking leverages code backdoor poisoning to protect dataset copyrights and prevent unauthorized training of NCMs. CoProtector [45] embeds watermarks at two levels: word-level, by modifying identifiers and injecting specific words into comments and code, and sentence-level, by inserting statements while embedding words into comments and code. This approach ensures effective protection for code search, summarization, and completion tasks. However, CoProtector lacks stealthiness, making it easily detectable by humans [44]. To address this, CodeMark [44] introduces a stealthier approach by injecting SPT (e.g., replacing “a += 1” with “a = a + 1”), forming learnable patterns in datasets. These patterns do not alter functionality but embed hidden backdoors in NCMs, enabling watermark detection in copyright disputes. This research explores vulnerabilities in code dataset watermarking and highlights the need to enhance dataset security.

8 Conclusion

In this paper, we reveal the relationship between watermark embedding rules and code across both the formal and natural channels, finding that code watermark patterns form specific distributions in code datasets. Building on this, we propose DeCoMa, a novel technique for detecting and purifying code dataset watermarks. We evaluate DeCoMa across 14 code watermark detection scenarios, and the results show that DeCoMa can effectively and efficiently detect code watermarks, significantly outperforming three baselines. In terms of effectiveness, DeCoMa achieves a 100% recall with nearly no impact on model training performance. In terms of efficiency, DeCoMa provides a speedup of 31.5 to 130.9× compared to the baselines.

9 Data Availability

Our source code and experimental data are available at [2].

Acknowledgement

This work is partially supported by the National Natural Science Foundation of China (U24A20337, 62372228), the Shenzhen-Hong Kong-Macau Technology Research Programme (Type C) (Grant No.SGDX20230821091559018), and the Fundamental Research Funds for the Central Universities (14380029).

References

- [1] Inc. Amazon Web Services. 2023. CodeWhisperer. site: <https://aws.amazon.com/codewhisperer/>.
- [2] Anonymous. 2024. DeCoMa. <https://anonymous.4open.science/r/DeCoMa-C00B>.
- [3] Genevieve Arboit. 2002. A Method for Watermarking Java Programs via Opaque Predicates. In *Proceedings of the 5th International Conference on Electronic Commerce Research*. ICECR, Montreal, Canada, 102–110.
- [4] Inc. Beijing Guixin Technology. 2022. aiXcoder. site: <https://www.aixcoder.com/>.
- [5] Casey Casalnuovo, Earl T. Barr, Santanu Kumar Dash, Prem Devanbu, and Emily Morgan. 2020. A theory of dual channel constraints. <https://dl.acm.org/doi/10.1145/3377816.3381720>. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*. Association for Computing Machinery, 25–28.
- [6] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T. Devanbu, and Baishakhi Ray. 2022. NatGen: generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Singapore, Singapore, 18–30. <https://doi.org/10.1145/3540250.3549162>
- [7] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian M. Molloy, and Biplav Srivastava. 2019. Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering. In *Workshop on Artificial Intelligence Safety 2019 co-located with the Thirty-Third AAAI Conference on Artificial Intelligence 2019 (AAAI-19) (CEUR Workshop Proceedings, Vol. 2301)*. CEUR-WS.org, Honolulu, Hawaii. https://ceur-ws.org/Vol-2301/paper_18.pdf
- [8] Jianping Chen, Kui Li, Wanzhi Wen, Weixu Chen, and Chenxue Yan. 2017. Software Watermarking for Java Program Based on Method Name Encoding. In *Proceedings of the International Conference on Advanced Intelligent Systems and Informatics 2017 (Advances in Intelligent Systems and Computing, Vol. 639)*. Springer, Cairo, Egypt, 865–874. https://doi.org/10.1007/978-3-319-64861-3_81
- [9] Christian S. Collberg and Clark D. Thomborson. 1999. Software Watermarking: Models and Dynamic Embeddings. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, San Antonio, TX, USA, 311–324. <https://doi.org/10.1145/292540.292569>
- [10] Chunrong Fang, Weisong Sun, Yuchen Chen, Xiao Chen, Zhao Wei, Qunjun Zhang, Yudu You, Bin Luo, Yang Liu, and Zhenyu Chen. 2024. Esale: Enhancing Code-Summary Alignment Learning for Source Code Summarization. *IEEE Trans. Software Eng.* 50, 8 (2024), 2077–2095. <https://doi.org/10.1109/TSE.2024.3422274>
- [11] Inc. GitHub. 2022. GitHub Copilot. site: <https://copilot.github.com/>.
- [12] Yoav Goldberg. 2014. word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method. <https://arxiv.org/abs/1402.3722>. *arXiv preprint arXiv:1402.3722* (2014).
- [13] Frank E Grubbs. 1969. Procedures for detecting outlying observations in samples. <https://doi.org/10.1080/00401706.1969.10490657>. *Technometrics* 11, 1 (1969), 1–21.
- [14] James Alexander George Hamilton and Sebastian Danicic. 2011. A survey of static software watermarking. In *2011 World Congress on Internet Security, WorldCIS 2011*. IEEE, London, UK, 100–107. <https://ieeexplore.ieee.org/document/5749891/>
- [15] Jingxuan He and Martin T. Vechev. 2023. Large Language Models for Code: Security Hardening and Adversarial Testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Copenhagen, Denmark, 1865–1879. <https://doi.org/10.1145/3576915.3623175>
- [16] Inc. Hugging Face. 2016. Hugging Face. site: <https://huggingface.co/>.
- [17] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *arXiv abs/1909.09436* (2019). *arXiv:1909.09436* <http://arxiv.org/abs/1909.09436>
- [18] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*. San Diego, CA, USA. <http://arxiv.org/abs/1412.6980>
- [19] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A Watermark for Large Language Models. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 17061–17084. <https://proceedings.mlr.press/v202/kirchenbauer23a/kirchenbauer23a.pdf>
- [20] Jorma Laurikkala, Martti Juhola, Erna Kentala, N Lavrac, S Miksch, and B Kavsek. 2000. Informal identification of outliers in medical data. In *Fifth international workshop on intelligent data analysis in medicine and pharmacology*, Vol. 1. 20–24.
- [21] Taehyun Lee, Seokhee Hong, Jaewoo Ahn, Ilgee Hong, Hwaran Lee, Sangdoo Yun, Jamin Shin, and Gunhee Kim. 2024. Who Wrote this Code? Watermarking for Code Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Bangkok, Thailand, 4890–4911. <https://aclanthology.org/2024.acl-long.268>
- [22] Jia Li, Zhuo Li, Huangzhao Zhang, Ge Li, Zhi Jin, Xing Hu, and Xin Xia. 2024. Poison Attack and Poison Detection on Deep Source Code Processing Models. *ACM Trans. Softw. Eng. Methodol.* 33, 3 (2024), 62:1–62:31. <https://doi.org/10.1145/3630008>

- [23] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* 2023 (2023). <https://openreview.net/forum?id=KoFOg41haE>
- [24] Yanzhou Li, Shangqing Liu, Kangjie Chen, Xiaofei Xie, Tianwei Zhang, and Yang Liu. 2023. Multi-target Backdoor Attacks for Code Pre-trained Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Toronto, Canada, 7236–7254. <https://doi.org/10.18653/V1/2023.ACL-LONG.399>
- [25] Yi Li, Aashish Yadavally, Jiaying Zhang, Shaohua Wang, and Tien N Nguyen. 2023. DeMinify: Neural Variable Name Recovery and Type Inference. <https://doi.org/10.1145/3611643.3616368>. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 758–770.
- [26] Fang Liu, Ge Li, Yunfei Zhao, and Zhi Jin. 2020. Multi-task Learning based Pre-trained Language Model for Code Completion. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Melbourne, Australia, 473–485. <https://doi.org/10.1145/3324884.3416591>
- [27] Haoyu Ma, Chunfu Jia, Shijia Li, Wantong Zheng, and Dinghao Wu. 2019. Xmark: Dynamic Software Watermarking Using Collatz Conjecture. *IEEE Trans. Inf. Forensics Secur.* 14, 11 (2019), 2859–2874. <https://doi.org/10.1109/TIFS.2019.2908071>
- [28] M.Brunsfeld, P.Thomson, A.Hlynskyi, J.Vera, P.Turnbull, T.Clem, D.Creager, A.Helwer, R.Rix, H.van Antwerpen, M.Davis, Ika, T.-A.Nguyen, S.Brunck, N.Hasabnis, bfredl, M.Dong, V.Panteleev, ikrima, S.Kalt, K.Lampe, A.Pinkus, M.Schmitz, M.Krupcale, narpfel, S.Gallegos, V.Martí, Edgar, and G.Fraser. 2020. Tree-sitter: An incremental parsing system for programming tools. <https://github.com/tree-sitter/tree-sitter>.
- [29] Tomas Mikolov. 2013. Efficient estimation of word representations in vector space. <https://doi.org/10.48550/arXiv.1301.3781>. *arXiv preprint arXiv:1301.3781* 3781 (2013).
- [30] Jens Palsberg, S. Krishnaswamy, Minseok Kwon, Di Ma, Qiuyun Shao, and Y. Zhang. 2000. Experience with Software Watermarking. In *Proceedings of the 16th Annual Computer Security Applications Conference*. IEEE Computer Society, New Orleans, Louisiana, USA, 308–316. <https://doi.org/10.1109/ACSAC.2000.898885>
- [31] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. ACL, Philadelphia, PA, USA, 311–318. <https://doi.org/10.3115/1073083.1073135>
- [32] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy*. IEEE, San Francisco, CA, USA, 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [33] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. <https://aclanthology.org/D14-1162/>. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
- [34] Steven T Piantadosi. 2014. Zipf’s word frequency law in natural language: A critical review and future directions. <https://doi.org/10.3758/s13423-014-0585-6>. *Psychonomic bulletin & review* 21 (2014), 1112–1130.
- [35] Pyminifie. 2014. Pyminifie. <https://github.com/liftoff/pyminifier>.
- [36] Goutham Ramakrishnan and Aws Albarghouthi. 2022. Backdoors in Neural Models of Source Code. In *Proceedings of the 26th International Conference on Pattern Recognition*. IEEE, Montreal, QC, Canada, 2892–2899. <https://doi.org/10.1109/ICPR56361.2022.9956690>
- [37] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *arXiv abs/2009.10297* (2020). <https://arxiv.org/abs/2009.10297>
- [38] Peter J Rousseeuw and Christophe Croux. 1993. Alternatives to the median absolute deviation. <https://doi.org/10.1080/01621459.1993.10476408>. *Journal of the American Statistical association* 88, 424 (1993), 1273–1283.
- [39] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *arXiv abs/2308.12950* (2023). <https://doi.org/10.48550/arXiv.2308.12950>
- [40] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, Anaheim, CA, USA, 2205–2222. <https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>
- [41] Weisong Sun, Yuchen Chen, Guanhong Tao, Chunrong Fang, Xiangyu Zhang, Qunjun Zhang, and Bin Luo. 2023. Backdoor Neural Code Search. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Toronto, Canada, 9692–9708. <https://doi.org/10.18653/V1/2023>

ACL-LONG.540

- [42] Weisong Sun, Chunrong Fang, Yuchen Chen, Guan hong Tao, Tingxu Han, and Qunjun Zhang. 2022. Code Search based on Context-aware Code Translation. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering*. ACM, May 25-27, 388–400. <https://doi.org/10.1145/3510003.3510140>
- [43] Weisong Sun, Chunrong Fang, Yuchen Chen, Qunjun Zhang, Guan hong Tao, Yudu You, Tingxu Han, Yifei Ge, Yuling Hu, Bin Luo, and Zhenyu Chen. 2024. An Extractive-and-Abstractive Framework for Source Code Summarization. *ACM Trans. Softw. Eng. Methodol.* 33, 3 (2024), 75:1–75:39. <https://doi.org/10.1145/3632742>
- [44] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. CodeMark: Imperceptible Watermarking for Code Datasets against Neural Code Completion Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, San Francisco, CA, USA, 1561–1572. <https://doi.org/10.1145/3611643.3616297>
- [45] Zhensu Sun, Xiaoning Du, Fu Song, Mingze Ni, and Li Li. 2022. CoProtector: Protect Open-Source Code against Unauthorized Training Usage with Data Poisoning. In *WWW '22: The ACM Web Conference 2022*. ACM, Virtual Event, Lyon, France, 652–660. <https://doi.org/10.1145/3485447.3512225>
- [46] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, Sydney, NSW, Australia, 3319–3328. <http://proceedings.mlr.press/v70/sundararajan17a.html>
- [47] Jeffrey Svajlenko, Judith F. Islam, Iman Keivanloo, Chanchal Kumar Roy, and Mohammad Mamun Mia. 2014. Towards a Big Data Curated Benchmark of Inter-project Code Clones. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution*. IEEE Computer Society, Victoria, BC, Canada, 476–480. <https://doi.org/10.1109/ICSME.2014.77>
- [48] Alexey Svyatkovskiy, Ying Zhao, Shengyu Fu, and Neel Sundaresan. 2019. Pythia: AI-assisted Code Completion System. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery*. ACM, Anchorage, AK, USA, 2727–2735. <https://doi.org/10.1145/3292500.3330699>
- [49] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv abs/2307.09288* (2023). <https://doi.org/10.48550/ARXIV.2307.09288> arXiv:2307.09288
- [50] Brandon Tran, Jerry Li, and Aleksander Madry. 2018. Spectral Signatures in Backdoor Attacks. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems*. Curran Associates, Montréal, Canada, 8011–8021. <https://proceedings.neurips.cc/paper/2018/hash/280cf18baf4311c92aa5a042336587d3-Abstract.html>
- [51] Peter D Turney and Patrick Pantel. 2010. From frequency to meaning: Vector space models of semantics. <https://arxiv.org/abs/1003.1141>. *Journal of artificial intelligence research* 37 (2010), 141–188.
- [52] Yao Wan, Jingdong Shu, Yulei Sui, Guandong Xu, Zhou Zhao, Jian Wu, and Philip S. Yu. 2019. Multi-modal Attention Network Learning for Semantic Source Code Retrieval. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, San Diego, CA, USA, 13–25. <https://doi.org/10.1109/ASE.2019.00012>
- [53] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, et al. 2022. Recode: Robustness evaluation of code generation models. <https://arxiv.org/abs/2212.10264>. *arXiv preprint arXiv:2212.10264* (2022).
- [54] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 8696–8708. <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685>
- [55] BenLong Wu, Kejiang Chen, Yanru He, Guoqiang Chen, Weiming Zhang, and Nenghai Yu. 2024. CodeWMBench: An Automated Benchmark for Code Watermarking Evaluation. In *ACM Turing Award Celebration Conference 2024*. ACM, Changsha, China. <https://doi.org/10.1145/3674399.3674447>
- [56] Borui Yang, Wei Li, Liyao Xiang, and Bo Li. 2024. SrcMarker: Dual-Channel Source Code Watermarking via Scalable Code Transformations. In *IEEE Symposium on Security and Privacy*. IEEE, San Francisco, CA, USA, 4088–4106. <https://doi.org/10.1109/SP54263.2024.00097>
- [57] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy Backdoor Attack for Code Models. *IEEE Trans. Software Eng.* 50, 4 (2024), 721–741. <https://doi.org/10.1109/TSE.2024.3361661>
- [58] Zhou Yang, Bowen Xu, Jie M Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy backdoor attack for code models. . *IEEE Transactions on Software Engineering* (2024).
- [59] Shiwen Yu, Ting Wang, and Ji Wang. 2022. Data augmentation by program transformation. <https://doi.org/10.1016/j.jss.2022.111304>. *Journal of Systems and Software* 190 (2022), 111304.
- [60] William Zhu, Clark D. Thomborson, and Fei-Yue Wang. 2005. A Survey of Software Watermarking. In *Intelligence and Security Informatics, IEEE International Conference on Intelligence and Security Informatics (Lecture Notes in Computer*

Science, Vol. 3495). Springer, Atlanta, GA, USA, 454–458. https://doi.org/10.1007/11427995_42

[61] G Zipf. 1936. *The Psychobiology of Language* London: Routledge. <https://doi.org/10.4324/9781315009421>. (1936).

Received 2025-02-27; accepted 2025-03-31